CSCI/MATH 485 Assignment

Customer Churn Prediction with XGBoost

Starter Notebook

This notebook is compatible with **Jupyter Notebook** and **Google Colab.**

This starter code is only to get you started. You can change any of the existing code here to complete all the tasks.

Complete all `TODO` sections. Make sure your final submission includes:

- data analysis,
- a tuned XGBoost model,
- your chosen main evaluation metric and justification,
- interpretation of top important features,
- and a final comparison with the baseline model.

1. Setup

In [1]:
```
# If you are using Google Colab, uncomment the next line if xgboost is not i
# !pip install xgboost
```

In [2]:
```
import pandas as pd
import numpy as np
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)

from xgboost import XGBClassifier
```

2. Load the Dataset

Instructions for students

- Load the IBM Telco Customer Churn dataset.
- Display the first few rows.
- Confirm the dataset shape.
- If the url doesn't work for you, download the csv file from the Canvas Assignment#4

In [3]:
```
#url = "https://raw.githubusercontent.com/plotly/datasets/master/telco-custo
file = open('telco-customer-churn-by-IBM.csv')
df = pd.read_csv(file)

# Display the first 5 rows of the dataset (done below)
df.head()

# TODO:
# Display the shape of the dataset
shape = df.shape
print(shape)
```

```
(7043, 21)
```

3. Data Exploration

Complete the following:

- Print all column names
- Show data types
- Count missing values in each column
- Show the distribution of the target variable
- Write a short note: Is this a classification or regression problem? Why is this useful in business?

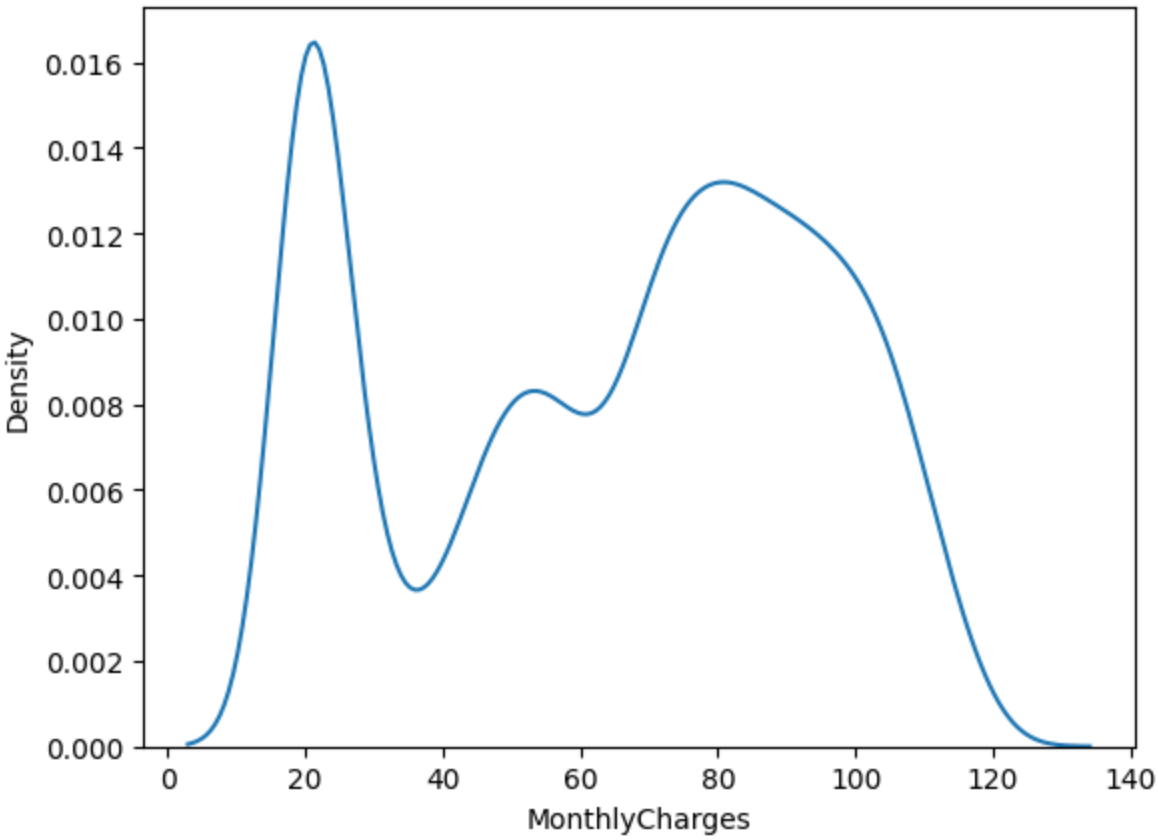
In [4]:
```
# TODO:
# Print column names
# Print data types
# Print missing values for each column
# Print value counts for the target column

print("Columns:")
# your code here
cols = df.columns
print(cols)

print("\nData types:")
# your code here
dtypes = df.dtypes
```

```
print(dtypes)

print("\nMissing values:")
# your code here
mVals = df.isnull() # .isna() ?
print(mVals)

print("\nTarget distribution:")
# yourcode here


tDist = df['Churn'].value_counts()

print(tDist)

# Extra: display other information of the dataset that you think can be usef
print("Monthly Charges density plot")
sns.kdeplot(data=df, x='MonthlyCharges')
#sns.kdeplot(data=df, x='tenure')
print("Contract values")
print(df['Contract'].value_counts())

print("Tenure quantiles:")
print(df['tenure'].quantile([0, 0.25, 0.5, 0.75, 1]))

print("Head:")
print(df.head())

print("Print all values for string variables:")
print(df.select_dtypes(include=['string']).apply(pd.Series.unique))

print("Print all values for PaymentMethod")
print(df['PaymentMethod'].unique())

print("Print all values for SeniorCitizen")
print(df['SeniorCitizen'].unique())
```

```
Columns:
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='str')

Data types:
customerID              str
gender                  str
SeniorCitizen         int64
Partner                 str
Dependents              str
tenure                int64
PhoneService            str
MultipleLines           str
InternetService         str
OnlineSecurity          str
OnlineBackup            str
DeviceProtection        str
TechSupport             str
StreamingTV             str
StreamingMovies         str
Contract                str
PaperlessBilling        str
PaymentMethod           str
MonthlyCharges      float64
TotalCharges            str
Churn                   str
dtype: object

Missing values:
      customerID  gender  SeniorCitizen  Partner  Dependents  tenure  \
0          False   False          False    False       False   False
1          False   False          False    False       False   False
2          False   False          False    False       False   False
3          False   False          False    False       False   False
4          False   False          False    False       False   False
...          ...     ...            ...      ...         ...     ...
7038       False   False          False    False       False   False
7039       False   False          False    False       False   False
7040       False   False          False    False       False   False
7041       False   False          False    False       False   False
7042       False   False          False    False       False   False

      PhoneService  MultipleLines  InternetService  OnlineSecurity  ...  \
0            False          False            False           False  ...
1            False          False            False           False  ...
2            False          False            False           False  ...
3            False          False            False           False  ...
4            False          False            False           False  ...
...            ...            ...              ...             ...  ...
7038         False          False            False           False  ...
7039         False          False            False           False  ...
7040         False          False            False           False  ...
```

```
7041         False          False            False           False  ...
7042         False          False            False           False  ...

      DeviceProtection  TechSupport  StreamingTV  StreamingMovies  Contract
\
0                False        False        False            False     False
1                False        False        False            False     False
2                False        False        False            False     False
3                False        False        False            False     False
4                False        False        False            False     False
...                ...          ...          ...              ...       ...
7038             False        False        False            False     False
7039             False        False        False            False     False
7040             False        False        False            False     False
7041             False        False        False            False     False
7042             False        False        False            False     False

      PaperlessBilling  PaymentMethod  MonthlyCharges  TotalCharges  Churn
0                False          False           False         False  False
1                False          False           False         False  False
2                False          False           False         False  False
3                False          False           False         False  False
4                False          False           False         False  False
...                ...            ...             ...           ...    ...
7038             False          False           False         False  False
7039             False          False           False         False  False
7040             False          False           False         False  False
7041             False          False           False         False  False
7042             False          False           False         False  False

[7043 rows x 21 columns]

Target distribution:
Churn
No     5174
Yes    1869
Name: count, dtype: int64
Monthly Charges density plot
Contract values
Contract
Month-to-month    3875
Two year          1695
One year          1473
Name: count, dtype: int64
Tenure quantiles:
0.00     0.0
0.25     9.0
0.50    29.0
0.75    55.0
1.00    72.0
Name: tenure, dtype: float64
Head:
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService
\
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
```

```
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling
\
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
Print all values for string variables:
customerID          [7590-VHVEG, 5575-GNVDE, 3668-QPYBK, 7795-CFOC...
gender                                                 [Female, Male]
Partner                                                     [Yes, No]
Dependents                                                  [No, Yes]
PhoneService                                                [No, Yes]
MultipleLines                            [No phone service, No, Yes]
InternetService                                [DSL, Fiber optic, No]
OnlineSecurity                          [No, Yes, No internet service]
OnlineBackup                            [Yes, No, No internet service]
DeviceProtection                        [No, Yes, No internet service]
TechSupport                             [No, Yes, No internet service]
StreamingTV                             [No, Yes, No internet service]
StreamingMovies                         [No, Yes, No internet service]
Contract                          [Month-to-month, One year, Two year]
PaperlessBilling                                            [Yes, No]
PaymentMethod       [Electronic check, Mailed check, Bank transfer...
TotalCharges        [29.85, 1889.5, 108.15, 1840.75, 151.65, 820.5...
Churn                                                       [No, Yes]
dtype: object
Print all values for PaymentMethod
<StringArray>
[         'Electronic check',              'Mailed check',
 'Bank transfer (automatic)',   'Credit card (automatic)']
Length: 4, dtype: str
Print all values for SeniorCitizen
[0 1]
```



TODO (write your answer below):

1. What kind of machine learning problem is this?
2. Why is churn prediction important in a business setting?

Replace this text with your answer.

I would go with a Bayesian Network, since it looks like the majority of the variables reduce to simple TRUE/FALSE results, which allow for the required conditional probabilities. I'm a little concerned at how I would incorporate the MonthlyCharges variable, since cost I would a priori expect cost to be a significant factor in customer's choice to end service.

Reducing churn when providing ongoing services leads to an increased customer base.

4. Basic Cleaning

Complet the following:

- Identify whether there is an ID column that should be removed
- Convert the target column into binary form if needed
- Convert any numeric-looking columns stored as text into numeric values

In [5]:
```
# Make a copy so the original data remains unchanged
df_clean = df.copy()

# TODO:
# 1. Drop any unnecessary identifier column(s)
# 2. Convert the target column to 0/1
# 3. Convert any columns that should be numeric into numeric type
# 4. Handle invalid parsing if needed

df_clean = df_clean.drop(columns=['customerID',
                                  'PaymentMethod',
                                  'MultipleLines',
                                  'Contract'])
# Yes/Nos
df_clean["Churn"] = df_clean["Churn"].map({"Yes": 1, "No": 0})
## Apparently I was overthinking things:
#df_clean["Partner"] = df_clean["Partner"].map({"Yes": 1, "No": 0})
#df_clean["PhoneService"] = df_clean["PhoneService"].map({"Yes": 1, "No": 0}
#df_clean["OnlineSecurity"] = df_clean["OnlineSecurity"].map({"Yes": 1, "No"
#df_clean["OnlineBackup"] = df_clean["OnlineBackup"].map({"Yes": 1, "No": 0}
#df_clean["DeviceProtection"] = df_clean["DeviceProtection"].map({"Yes": 1,

#df_clean["TechSupport"] = df_clean["TechSupport"].map({"Yes": 1, "No": 0})
#df_clean["StreamingTV"] = df_clean["StreamingTV"].map({"Yes": 1, "No": 0})
#df_clean["StreamingMovies"] = df_clean["StreamingMovies"].map({"Yes": 1, "N
#df_clean["PaperlessBilling"] = df_clean["PaperlessBilling"].map({"Yes": 1,
#df_clean["Dependents"] = df_clean["Dependents"].map({"Yes": 1, "No": 0})


df_clean['TotalCharges'] = pd.to_numeric(df_clean['TotalCharges'], errors="c
#df_clean[""] = df_clean[""].map({"Yes": 1, "No": 0})

# df_clean = df_clean.drop(columns=[...])
# df_clean["Churn"] = df_clean["Churn"].map({"Yes": 1, "No": 0})
# df_clean["TotalCharges"] = pd.to_numeric(df_clean["TotalCharges"], errors=
print(df_clean['TotalCharges'].dtype)
df_clean.head()
```

```
float64
```

Out[5]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	InternetService	Onlin
0	Female	0	Yes	No	1	No	DSL	
1	Male	0	No	No	34	Yes	DSL	
2	Male	0	No	No	2	Yes	DSL	
3	Male	0	No	No	45	No	DSL	
4	Female	0	No	No	2	Yes	Fiber optic	

5. Define Features and Target

Complet the following:

- Define `X` and `y`
- Set the correct target column

In [6]:
```
# TODO:
# Replace with the correct target column name
target_col = "Churn"

X = df_clean.drop(columns=[target_col])
y = df_clean[target_col]

print("Feature matrix shape:", X.shape)
print("Target shape:", y.shape)
```

```
Feature matrix shape: (7043, 16)
Target shape: (7043,)
```

6. Identify Numeric and Categorical Features

Complet the following:

- Create a list of numeric columns
- Create a list of categorical columns

In [7]:
```
# TODO:
# Identify numeric and categorical feature columns

numeric_features = ['MonthlyCharges',
                    'TotalCharges',
                    'tenure']

categorical_features = ['gender',
                        'SeniorCitizen',
                        'Partner',
                        'Dependents',
                        'PhoneService',
                        'InternetService',
                        'OnlineSecurity',
                        'OnlineBackup',
                        'DeviceProtection',
                        'TechSupport',
                        'StreamingTV',
                        'StreamingMovies',
                        'PaperlessBilling']

print("Numeric features:")
print(numeric_features)

print("\nCategorical features:")
print(categorical_features)
```

```
Numeric features:
['MonthlyCharges', 'TotalCharges', 'tenure']

Categorical features:
['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'PhoneService', 'Intern
etService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSuppo
rt', 'StreamingTV', 'StreamingMovies', 'PaperlessBilling']
```

7. Train/Test Split

Complet the following:

- Split the dataset into training and testing sets
- Use stratification if appropriate

In [8]:
```
# TODO:
# Create train/test split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

```
X_train shape: (5634, 16)
X_test shape: (1409, 16)
```

8. Preprocessing Pipelines

Build preprocessing for:

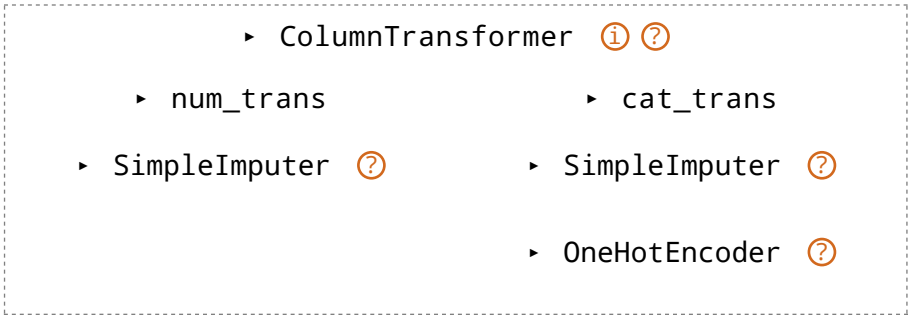
- numeric features
- categorical features

Then combine them into a `ColumnTransformer`.

In [9]:
```
# TODO:
# Build numeric preprocessing pipeline
numeric_transformer = Pipeline(steps=[
    # Example:
    # ("imputer", SimpleImputer(strategy="median"))
    ("num_imputer", SimpleImputer(strategy="median"))
])

# TODO:
# Build categorical preprocessing pipeline
categorical_transformer = Pipeline(steps=[
```

```
    # Example:
    # ("imputer", SimpleImputer(strategy="most_frequent")),
    # ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ("cat_impputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# TODO:
# Combine both preprocessing pipelines
preprocessor = ColumnTransformer(transformers=[
    # Example:
    # ("num", numeric_transformer, numeric_features),
    # ("cat", categorical_transformer, categorical_features)
    ("num_trans", numeric_transformer, numeric_features),
    ("cat_trans", categorical_transformer, categorical_features)
])

preprocessor
```

Out[9]:



9. Baseline Model: Logistic Regression

Complet the following:

- Train a Logistic Regression model as the baseline
- Generate predictions
- Evaluate using multiple metrics
- You may need to adjust `max_iter` if the model is not converging.

In [10]:

```
baseline_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=5000))
])

# TODO:
# Fit the baseline model
# Generate predicted labels
# Generate predicted probabilities

# your code here

baseline_model.fit(X_train, y_train)
baseline_pred = baseline_model.predict(X_test)
baseline_prob = baseline_model.predict_proba(X_test)[:, 1]
```

```
#baseline_pred_30 =
```

In [11]:
```
# TODO:
# Compute and print:
# - Accuracy
print("Accuracy:")
print(accuracy_score(y_test, baseline_pred))
print("0.1:", accuracy_score(y_test, (baseline_prob > 0.1).astype(int)))
print("0.2:", accuracy_score(y_test, (baseline_prob > 0.2).astype(int)))
print("0.3:", accuracy_score(y_test, (baseline_prob > 0.3).astype(int)))
print("0.4:", accuracy_score(y_test, (baseline_prob > 0.4).astype(int)))
print("0.5:", accuracy_score(y_test, (baseline_prob > 0.5).astype(int)))
print("0.6:", accuracy_score(y_test, (baseline_prob > 0.6).astype(int)))
print("0.7:", accuracy_score(y_test, (baseline_prob > 0.7).astype(int)))


# - Precision
print("Precision:")
print(precision_score(y_test, baseline_pred))
# - Recall
print("Recall:")
print(recall_score(y_test, baseline_pred))
# - F1-score
print("F1-Score:")
print(f1_score(y_test, baseline_pred))
print("0.1:", f1_score(y_test, (baseline_prob > 0.1).astype(int)))
print("0.2:", f1_score(y_test, (baseline_prob > 0.2).astype(int)))
print("0.3:", f1_score(y_test, (baseline_prob > 0.3).astype(int)))
print("0.4:", f1_score(y_test, (baseline_prob > 0.4).astype(int)))
print("0.5:", f1_score(y_test, (baseline_prob > 0.5).astype(int)))
print("0.6:", f1_score(y_test, (baseline_prob > 0.6).astype(int)))
print("0.7:", f1_score(y_test, (baseline_prob > 0.7).astype(int)))

# - ROC-AUC
print("ROC-AUC:")
print(roc_auc_score(y_test, baseline_prob))

# Suggested variable names:
# baseline_preds
# baseline_probs

# your code here
```

```
Accuracy:
0.7963094393186657
0.1: 0.5848119233498935
0.2: 0.6919801277501775
0.3: 0.7501774308019872
0.4: 0.794889992902768
0.5: 0.7963094393186657
0.6: 0.7899219304471257
0.7: 0.7721788502484032
Precision:
0.639871382636656
Recall:
0.5320855614973262
F1-Score:
0.581021897810219
0.1: 0.5489591364687741
0.2: 0.5890151515151515
0.3: 0.6097560975609756
0.4: 0.6299615877080665
0.5: 0.581021897810219
0.6: 0.4896551724137931
0.7: 0.3408624229979466
ROC-AUC:
0.8363558862280089
```

In [12]:
```
# Optional but helpful
# TODO:
# Print classification report and confusion matrix

# your code here
print("Classification Report:")
print(classification_report(y_test, baseline_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, baseline_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.89      0.87      1035
           1       0.64      0.53      0.58       374

    accuracy                           0.80      1409
   macro avg       0.74      0.71      0.72      1409
weighted avg       0.79      0.80      0.79      1409

Confusion Matrix:
[[923 112]
 [175 199]]
```

10. Choose Your Main Evaluation Metric

Choose **one main metric** for this churn problem.

You must explain:

- which metric you chose,
- why it is appropriate,
- and why it is more informative than accuracy alone for this problem.

F1-score seems like the best metric. It's focused to what we are concerned about (Churners), while weighing performance towards non-churners less. Though the default threshold of 0.5 seems too high, 0.4 looks to be better. It is more informative than Accuracy alone because Accuracy is including how often the model correctly identifies non-churners, who are the vast majority, which will give an inflated metric of performance. It will be technically more correct, but we don't care about the kind of correct it is; much the same as a model which simply "predicts" every integer as "not Prime" will have >95% Accuracy.

TODO (write your answer below):

Replace this text with your answer.

11. XGBoost Model

Complet the following:

- Build an XGBoost pipeline
- Tune at least 3 hyperparameters
- Use either `GridSearchCV` or your own tuning approach

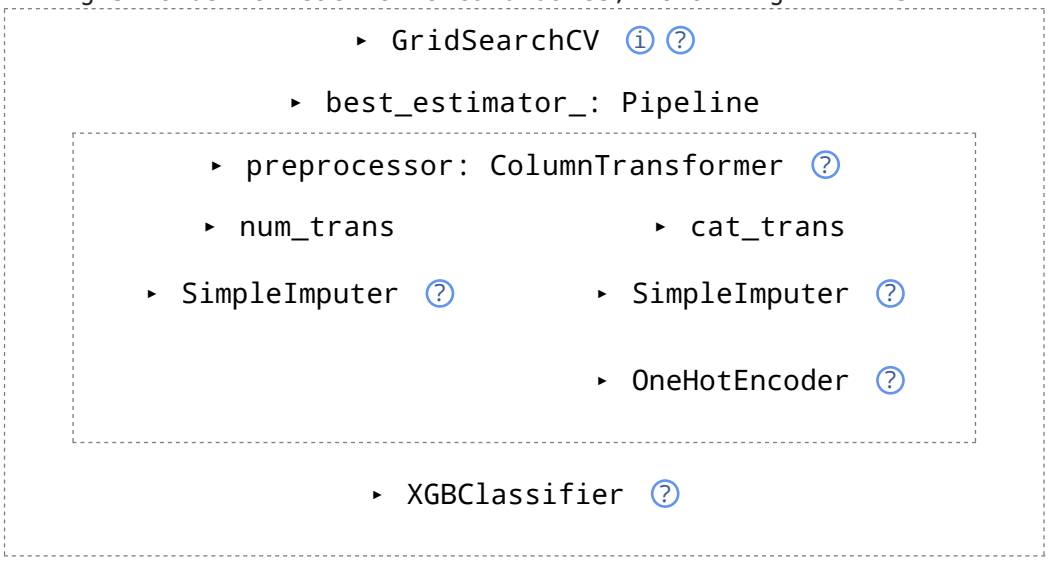
In [13]:
```
xgb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", XGBClassifier(
        eval_metric="logloss",
        random_state=42
    ))
])

# TODO:
# Define a hyperparameter grid with at least 3 hyperparameters
param_grid = {
    # Example format:
    # "classifier__n_estimators": [50, 100],
    # "classifier__max_depth": [3, 5],
    # "classifier__learning_rate": [0.05, 0.1]
    "classifier__n_estimators": [50, 100],
    "classifier__max_depth": [3, 5],
    "classifier__learning_rate": [0.05, 0.1]
}

param_grid
```

Out[13]:
```
{'classifier__n_estimators': [50, 100],
 'classifier__max_depth': [3, 5],
 'classifier__learning_rate': [0.05, 0.1]}
```

In [14]:
```
# TODO:
# Run GridSearchCV (or perform manual tuning)
# Choose a scoring metric that matches your selected main evaluation metric

grid_search = GridSearchCV(
    estimator=xgb_model,
    param_grid=param_grid,
    scoring="f1",   # TODO: replace with your chosen scoring metric
    cv=3,
    n_jobs=-1,
    verbose=1
)

# TODO:
# Fit grid search on training data
grid_search.fit(X_train, y_train)
# your code here
```

```
Fitting 3 folds for each of 8 candidates, totalling 24 fits
```

Out[14]:



In [15]:
```
# TODO:
# Print the best hyperparameters
# Save the best model
print(grid_search.best_params_)
# your code here
xgb_model = grid_search.best_estimator_
```

```
{'classifier__learning_rate': 0.1, 'classifier__max_depth': 5, 'classifier__
n_estimators': 100}
```

12. Evaluate the Tuned XGBoost Model

Evaluate XGBoost using the same metrics as the baseline.

In [16]:
```
# TODO:
# Generate predictions and probabilities using the best XGBoost model

# Suggested variable names:
# xgb_preds
# xgb_probs

# your code here
xgb_pred = xgb_model.predict(X_test)
xgb_prob = xgb_model.predict_proba(X_test)[:, 1]
threshold = 0.4
xgb_pred_th = (xgb_prob > threshold).astype(int)
```

In [17]:
```
# TODO:
# Compute and print:
# - Accuracy
# - Precision
# - Recall
# - F1-score
# - ROC-AUC

# your code here
print("Accuracy:")
print(accuracy_score(y_test, xgb_pred))
print(accuracy_score(y_test, xgb_pred_th))
# - Precision
print("Precision:")
print(precision_score(y_test, xgb_pred))
print(precision_score(y_test, xgb_pred_th))
# - Recall
print("Recall:")
print(recall_score(y_test, xgb_pred))
print(recall_score(y_test, xgb_pred_th))
# - F1-score
print("F1-Score:")
print(f1_score(y_test, xgb_pred))
print(f1_score(y_test, xgb_pred_th))
# - ROC-AUC
print("ROC-AUC:")
print(roc_auc_score(y_test, xgb_prob))
```

```
Accuracy:
0.7970191625266146
0.7913413768630234
Precision:
0.6527777777777778
0.5980392156862745
Recall:
0.5026737967914439
0.6524064171122995
F1-Score:
0.56797583081571
0.6240409207161125
ROC-AUC:
0.832925159523625
```

In [18]:
```
# Optional but helpful
# TODO:
# Print classification report and confusion matrix

# your code here
print("Classification Report:")
print(classification_report(y_test, xgb_pred))
print("Threshold: ", threshold)
print(classification_report(y_test, xgb_pred_th))
print("Confusion Matrix:")
print(confusion_matrix(y_test, xgb_pred))
print("Threshold: ", threshold)
print(confusion_matrix(y_test, xgb_pred_th))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.90      0.87      1035
           1       0.65      0.50      0.57       374

    accuracy                           0.80      1409
   macro avg       0.74      0.70      0.72      1409
weighted avg       0.79      0.80      0.79      1409

Threshold:  0.4
              precision    recall  f1-score   support

           0       0.87      0.84      0.86      1035
           1       0.60      0.65      0.62       374

    accuracy                           0.79      1409
   macro avg       0.73      0.75      0.74      1409
weighted avg       0.80      0.79      0.79      1409

Confusion Matrix:
[[935 100]
 [186 188]]
Threshold:  0.4
[[871 164]
 [130 244]]
```

13. Feature Importance

Use the trained XGBoost model to:

- extract feature importances,
- recover transformed feature names,
- display the top 5 to 10 most important features.

In [19]:
```
# TODO:
# Access the fitted preprocessor and fitted XGBoost classifier from the pipe

# Example structure:
```

```
# fitted_preprocessor = best_model.named_steps["preprocessor"]
# fitted_xgb = best_model.named_steps["classifier"]

# your code here

fitted_preprocessor = xgb_model.named_steps["preprocessor"]
fitted_xgb = xgb_model.named_steps["classifier"]
```

In [20]:
```
# TODO:
# Get transformed feature names
# Get feature importances
# Create a DataFrame sorted by importance

# Example structure:
# feature_names = fitted_preprocessor.get_feature_names_out()
# importances = fitted_xgb.feature_importances_

# your code here
feature_names = fitted_preprocessor.get_feature_names_out()
importances = fitted_xgb.feature_importances_
```

In [21]:
```
# TODO:
# Display the top 10 most important features
top_features = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
}).sort_values("importance", ascending=False)
# your code here
print(top_features.head(10))
```

```
                                  feature  importance
16            cat_trans__OnlineSecurity_No    0.287828
13          cat_trans__InternetService_DSL    0.166957
25              cat_trans__TechSupport_No    0.137715
14  cat_trans__InternetService_Fiber optic    0.127088
2                         num_trans__tenure    0.050663
34          cat_trans__PaperlessBilling_No    0.020458
30              cat_trans__StreamingTV_Yes    0.016511
0                num_trans__MonthlyCharges    0.016164
11              cat_trans__PhoneService_No    0.016020
9                 cat_trans__Dependents_No    0.015479
```

14. Interpret the Top Features

Write a short interpretation of the most important features.

Your explanation should:

- use plain language,
- connect features to churn behavior,
- and explain what the company might learn from them.

TODO (write your answer below):

The highest scoring feature that my model found was if the customer did not have the Online Security feature, followed by having DSL service, not having Tech Support, and the customer's Tenure to round out the top five. Interpreting these first five: I would group the OnlineSecurity and Techsupport as being specific features, likely with the most explanatory features. DSL and FiberOptic I would scrutinize as them both being included together since, if I were to continue the data science lifecycle, I strongly suspect this is a case of most customers have one or the other service. Such that most cases of churn would include them, simply because most of all cases include them. If that is not the case, perhaps people only using their non-ISP (telephone?) services are far more satisfied with what they recieved. Tenure would also need to be looked in order to have more explanitory power. However it being so high suggests that it does hold predictive power for Churn, the exact pattern of that would need to be looked at however. That said, since Churn is a matter of customers leaving, we would expect most of the low Tenure customers to be included in the Churn. So that would need to be accounted for, at which point it may lose any explanitory power.

The following five highest scoring features are: not having Paperless Billing, StreamingTV service, the customer's Monthly Charges, not having Phone Service, and not having Dependents. Interpreting the following five: PaperlessBilling is similar to OnlineSecurity and TechSupport in that it is more of a utility oriented, non-entertainment services provided. StreamingTV yes is countertuitive, but could suggest that it is currently priced higher than customers are willing. Not having a PhoneService could reinforce the second case I gave for the DSL/FiberOptic case, but a more intuitive explanation is simply that customers who would pay for phone service are likely older and thus in a more stable place. The customer Not having a Dependent may similarly suggest a more economically stabalized customer. Or at least one who is more reliant on having consistent internet service for work and or schooling needs.

The broad takeaways I would pass onto the business is customers who were not using their more Quality of Life services (OnlineSecurity, Techsuport, and PaperlessBilling) were more likely to Churn. Customers who were paying for Television Streaming services showed in increased risk of churn, whuch could be for a myriad of factors and worth looking into. Having a phone line and Dependents indicated customers who were less likely to churn, so looking into increasing advertising to those demographics could result in more customers with decreased likelihood to Churn.

15. Final Comparison: Logistic Regression vs XGBoost

Compare the two models using your results.

Your discussion should answer:

- Which model performed better?
- On which metric(s)?
- Why might XGBoost perform better on this dataset?
- What is one limitation of XGBoost?

In [22]:
```
# TODO:
# Print a side-by-side comparison of the main metrics
# for Logistic Regression and XGBoost
print("metric F1")
# your code here

print("Logistic Regression")
print("Threshold Default:", f1_score(y_test, baseline_pred))
print("Threshold 0.4:", f1_score(y_test, (baseline_prob > 0.4).astype(int)))


print("XGBoost")
print("Threshold Default:", f1_score(y_test, xgb_pred))
print("Threshold 0.4:", f1_score(y_test, xgb_pred_th))
```

```
metric F1
Logistic Regression
Threshold Default: 0.581021897810219
Threshold 0.4: 0.6299615877080665
XGBoost
Threshold Default: 0.56797583081571
Threshold 0.4: 0.6240409207161125
```

TODO (write your answer below):

The basic Logistic Regression performed better on the F1-Score metric. I suspect that XDGboost did worse mostly as a result of overfitting. I was trying to remove as few variables as possible, which added to the complexity. Given that Logistic Regression is less susceptible to this, I suspect that is is the source of the unexpected outcome. That possibility of overfitting is the limitation that I would hand to XGBoost.

16. Suggested Submission Checklist

Before submitting, make sure your notebook includes:

- completed code cells,
- outputs for each section,
- your selected main evaluation metric and justification,
- tuned XGBoost hyperparameters,
- feature importance results,
- interpretation of important features,

- and a final comparison of the two models.

In [ ]: